



University of Copenhagen  
MSc. Physics

# High Performance Parallel Computing

---

## Assignment 4

---

Author:

Benjamin Tariq Siddique

Contact: [vhb925@alumni.ku.dk](mailto:vhb925@alumni.ku.dk)

[ben123sid@gmail.com](mailto:ben123sid@gmail.com)

---

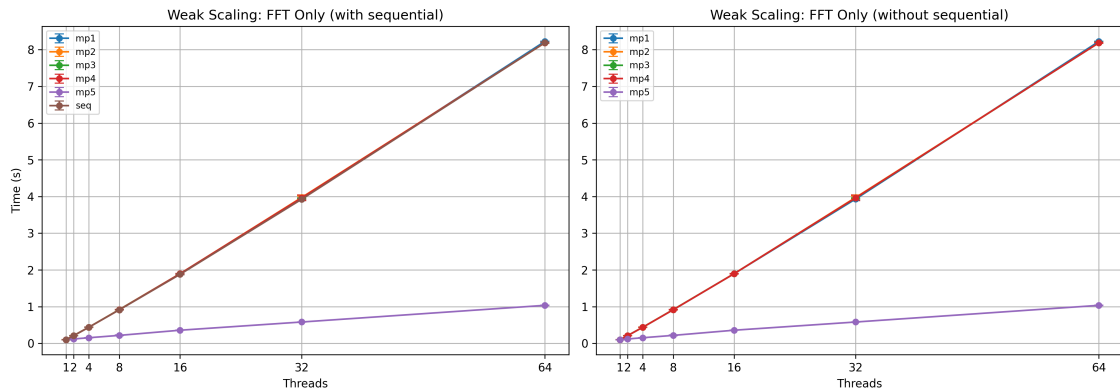
## 1 – OpenMP parallelise the program

| File name                                                    | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>seismogram_omp_independent_regions.cpp</code><br>(mp1) | Placed <code>#pragma omp parallel for</code> before almost every for loop in the <code>propagator</code> function. The only exceptions to this is the for-loop before <code>mean_wave = mean_wave / nsamp</code> , since it requires a pragma that can increment a shared <code>mean_wave</code> , and the inner loop of the nested for-loop later in the function. I expect this to make the code run significantly faster because a lot of the code is made up of simple for-loops making independent operations, allowing for significant speedup through the use of more cores. Though some of the pragmas might have a negligible or the opposite effect due to the overhead cost of executing them.                                                                                                                                                     |
| <code>seismogram_omp_fused_regions.cpp</code> (mp2)          | Unified all the pragmas before the for-loops under a single <code>parallel</code> -bracket. This allows for the use of a few <code>nowait</code> commands after <code>#pragma omp for</code> , granting greater control of the implicit barriers of the pragmas. The <code>nowaits</code> are justified by the fact that the cores will run into an implicit barrier, before the values that are calculated in that were calculated in that for-loop are used again. We should however also be cautious and implement <code>#pragma omp single</code> , when only a single core should be used (such as in the case described previously with <code>mean_wave</code> , where we dont want multiple cores to increment a shared value at the same time). This may possibly speedup the program, depending on the total size of the problem and overhead costs. |

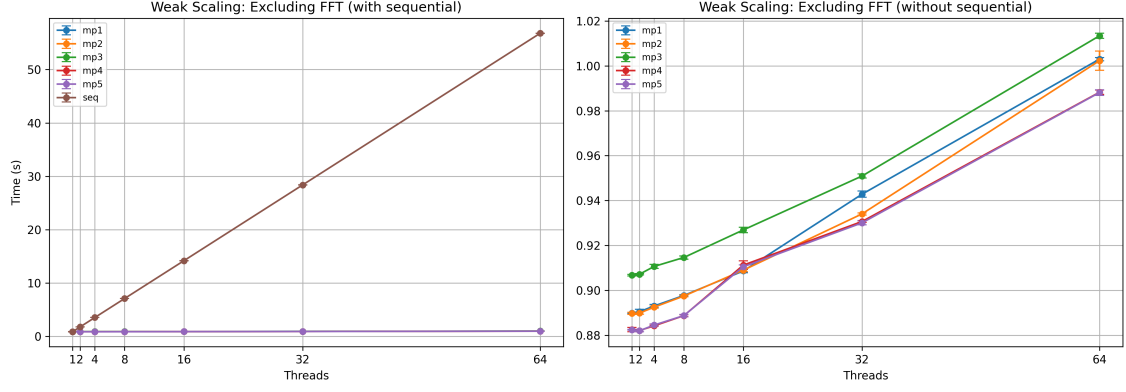
|                                                 |                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>seismogram_omp_reduction.cpp</code> (mp3) | Fixed the problem of multiple cores incrementing a shared value, by using <code>#pragma omp reduction(mean_wave: +)</code> , which allows each core to create a dummy variable, that is in the end incremented to the final shared value <code>mean_wave</code> . I would expect this to speed up the code, but once again it would depend on the size of the problem. |
| <code>seismogram_omp_SIMD.cpp</code> (mp4)      | Replaced every instance of <code>pragma omd for</code> with <code>#pragma omd for simd</code> , allowing each core to use single instruction, multiple data. This was permissable, because as stated previously, these are simple for-loops performing independent operations. Here great speedup is expected for obvious reasons.                                     |
| <code>seismogram_omp_FFT.cpp</code> (mp5)       | Split the FFT-function up using task pragmas such that we are computing <code>fft(odd)</code> and <code>fft(even)</code> simultaneously now. This should result in a lot of speedup in the code, since we are performing long recursive calculations at the same time.                                                                                                 |

**Table 1:** Table describing changes in implementations in chronological order of development. Each implementation is thus an upgrade of the previous one. Implementations saw variations in checksums on the 6th-7th decimals, but since this error was also present when running the sequential code with multiple cores I concluded, that it could be attributed to floating point errors and did not take it seriously.

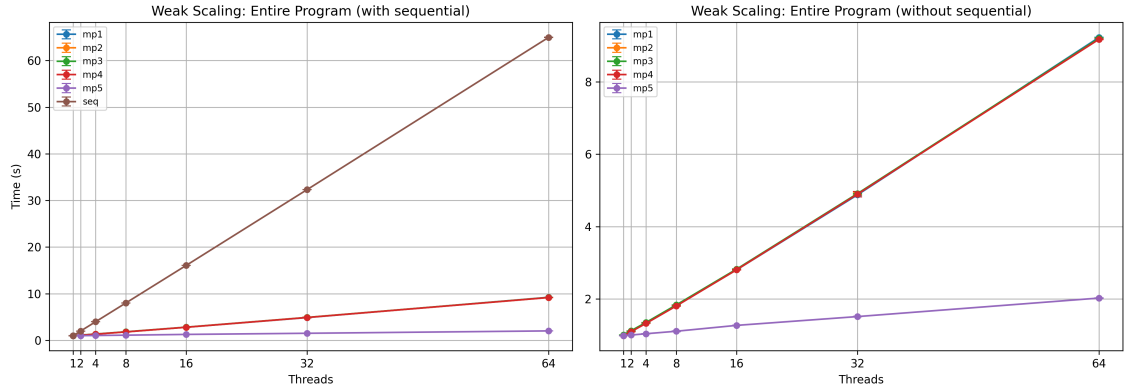
## 2 – Weak scaling using SLURM



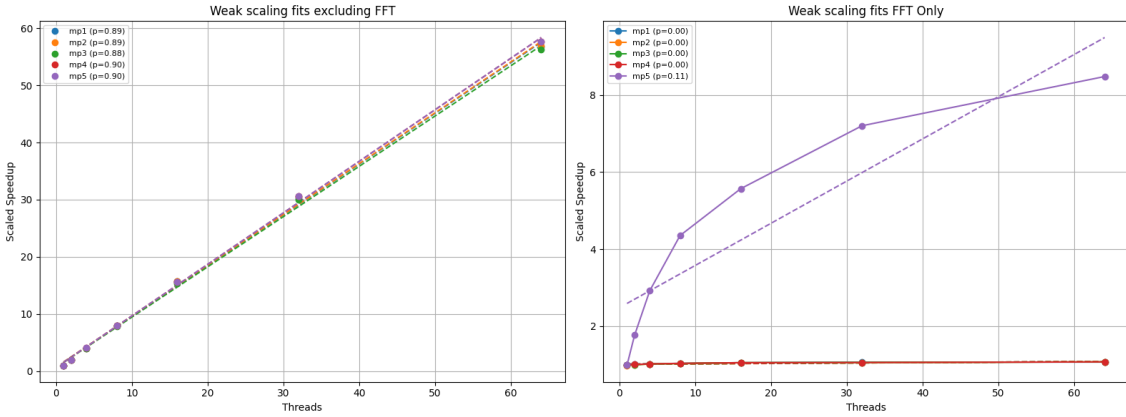
**Figure 1:** Total wall clock time spent doing FFT across implementations. On the left-hand-plot the sequential benchmark is included, while on the right-hand-side it is excluded. Note that the graphs for mp1-4 overlap.



**Figure 2:** Total wall clock time spent executing the program without the FFT-part across implementations. On the left-hand-plot the sequential benchmark is included, while on the right-hand-side it is excluded.



**Figure 3:** Total wall clock time spent executing the entire program across implementations. On the left-hand-plot the sequential benchmark is included, while on the right-hand-side it is excluded. Note that the graphs for mp1-4 overlap.



**Figure 4:** Speedup of the different implementations compared to the sequential code. On the left-hand-side we fit the linear speed up of the non-FFT code, while the right-hand-side measures the speedup of the  $N \log(N)$  FFT function. A linear fit was attempted on the RHS, but obviously does not extrapolate properly for mp5.

As a general comment on these figures, the uncertainties in runtimes were incredibly small and barely visible on the graphs. All implementations were scaled using  $N_{freq} = 65536 \cdot \text{cores}$ .

Let us first discuss fig. 1, which shows us the parallelising benefits from the FFT function. Since

we didn't touch it in mp1-4, their graphs obviously overlap with the sequential code. However mp5 scales much better than previous implementations, through its efficient use of the task pragmas, allowing it to run the FFT calculations simultaneously. This speedup was expected.

Moving on to fig. 2, which compares the non-FFT part of the code. Here we see that we've almost entirely removed the main bottle-neck of the program by using simple for pragmas (which is clearly visible on the mp1-graph). This makes sense since a large part of the code is just simple for-loops. Interestingly, the mp3 implementation, where we added the reduction clause is slower than other implementations, suggesting that the parallelization is not worth the overhead costs. Otherwise we see that mp4 and mp5, which are essentially on top of each other, provide the best performance, suggesting that SIMD (Main characteristic of the mp4-implementation) provides small but noticeable improvement. Unifying the separated parallel regions into a single region (mp2), also yielded a small speedup.

In fig. 3 we compare the entire programs runtimes. Once again mp1-4 fully overlap, because they are all delayed by the major bottleneck that is the FFT-function. We see here clearly the benefits of employing pragmas in the FFT-function, since mp5 performs much better, than all other implementations, being more than 30x faster than the sequential code.

In the final fig. 4 we fitted on the LHS using Gustafsons weak scaling law. I.e. if the problem grows linearly with the number of cores we can describe its speedup through:

$$\text{speedup} = s + p \cdot (1 - s) = \frac{T_{seq} \cdot N}{T_N}$$

Where  $p$  is the parallel fraction of the code,  $T_{seq}$  is the time taken for a sequentialized run,  $N$  is the number of cores used and  $T_N$  is the time taken for a parallelized run. For the RHS plot we know that the FFT algorithm has time complexity  $N_{freqs} \log(N_{freqs})$ , meaning we can modify the equation to:

$$\text{speedup} = s + p \cdot (1 - s) = \frac{T_{seq} \cdot N_{freqs} \log(N_{freqs})}{T_N \cdot N_{freqs}[1] \log(N_{freqs}[1])}$$

Where  $N_{freqs}[1]$  essentially represents the size of the problem for a single core. On the LHS we see a clear linear connection with high parallelization fractions (0.88 - 0.90) across all implementations(though largest for mp4 and mp5, which both employ SIMD), while on the RHS we clearly see more logarithmic growth of the speedup (ignoring mp1-4, since we didnt change the FFT-function here). The RHS is trivially explained once one realizes  $N_{freq} = 65536 \cdot N$ , so there is a logarithmic relationship between cores and speedup and the law doesn't apply properly.

## 3 – Code

### 3.1 seismogram\_omp\_independent\_regions.cpp

```
/*
Assignment: Make an OpenMP parallelised wave propagation
```

```

    model for computing the seismic repsonse for a wave
    propagating through a horizontally stratified medium
*/
#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <chrono>
#include <thread>
#include <complex>
#include <cmath>
#include <omp.h>

// shorthand name for complex number type definition below
typedef std::complex<double> Complex;

// =====
// The number of frequencies sets the cost of the problem
#ifndef NFREQ
#define NFREQ (64*1024)
#endif
const long nfreq = NFREQ; // frequencies in spectrum

// =====
// Initialize Basic Constants
const double dT=0.001;    // sampling distance
const long nsamp=2*nfreq; // samples in seismogram
double dF = 1/(nsamp*dT); // Frequency resolution (frequency sampling
                           distance)

// =====
// Custom memory allocator for NUMA-aware first touch memory placement
template <class T> class NUMA_Allocator {
public:
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef T& reference;
    typedef const T& const_reference;
    typedef size_t size_type;
    typedef T value_type;

    NUMA_Allocator() { }
    NUMA_Allocator(const NUMA_Allocator& _r) { }
    ~NUMA_Allocator() { }

```

```

// allocate raw memory including page placement
pointer allocate(size_t numObjects, const void *localityHint=0) {
    size_t len = numObjects * sizeof(T);
    void *m = std::malloc(len);
    char *p = static_cast<char*>(m);
    if(!omp_in_parallel()) {
#pragma omp parallel for schedule(static)
        for(size_t i=0; i<len; i+=sizeof(T)) {
            for(size_t j=0; j<sizeof(T); ++j) {
                p[i+j]=0;
            }
        }
    }

    return static_cast<T*>(m);
}

// free raw memory
void deallocate(pointer ptrToMemory, size_t numObjects) {
    std::free(ptrToMemory);
}

// construct object at given address
void construct(pointer p, const value_type& x) {
    new(p) value_type(x);
}

// destroy object at given address
void destroy(pointer p) {
    p->~value_type();
}

private:
    void operator=(const NUMA_Allocator&) {}
};

// =====
// shorthand names for vector types
#ifdef NUMA_ALLOCATOR
// Use NUMA-aware first touch allocator
typedef std::vector<Complex, NUMA_Allocator<Complex>> ComplexVector;
typedef std::vector<double, NUMA_Allocator<double>> DoubleVector;
#else
// Use standard allocator
typedef std::vector<Complex> ComplexVector;

```

```

typedef std::vector<double> DoubleVector;
#endif

// read data file with one number per line
DoubleVector read_txt_file(std::string fname) {
    DoubleVector data; // vector of data points
    std::string line; // string to read in each line
    std::ifstream file(fname); // open file
    while (std::getline(file, line)) // loop over lines until end of
        file
        data.push_back(std::stod(line)); // convert string to double and
        store in array
    return data;
}

// Cooley Tukey FFT (in-place computation)
template<typename VectorType>
void fft(VectorType& x)
{
    const long N = x.size();
    if (N <= 1) return;

    // divide
    std::vector<Complex> even(N/2), odd(N/2);
    for (long i=0; i<N/2; i++) {
        even[i] = x[2*i];
        odd[i] = x[2*i+1];
    }

    // conquer
    fft(even);
    fft(odd);

    // combine
    for (long k = 0; k < N/2; k++)
    {
        Complex t = std::polar(1.0, -2 * M_PI * k / N) * odd[k];
        x[k] = even[k] + t;
        x[k+N/2] = even[k] - t;
    }
}

// inverse fft (in-place)
template<typename VectorType>
void ifft(VectorType& x)

```

```

{
    double inv_size = 1.0 / x.size();
    for (auto& xx: x) xx = std::conj(xx); // conjugate the input
        fft(x); // forward fft
    for (auto& xx: x)
        xx = std::conj(xx) // conjugate the output
            * inv_size; // scale the numbers
}

// Main routine: propagate wave through layers and compute seismogram
DoubleVector propagator(DoubleVector wave,
                        DoubleVector density,
                        DoubleVector velocity) {
    const long nlayers = density.size();
    DoubleVector imp(nlayers); // impedance
    DoubleVector ref(nlayers-1); // reflection coefficient
    ComplexVector half_filter(nfreq/2+1,1); // half filter
    ComplexVector filter(nfreq+1); // full filter
    DoubleVector half_wave(nfreq+1,0); // half wave
    ComplexVector wave_spectral(nsamp); // FFT(wave)
    ComplexVector U(nfreq+1,0); // Upgoing waves
    ComplexVector Upad(nsamp,0); // FFT(seismogram)
    DoubleVector seismogram(nsamp); // final seismogram
    long n_wave = wave.size(); // size of wave array
    long lc = std::lround(std::floor(nfreq*0.01)); // low-cut indices
    double mean_wave = 0.; // wave zero point
    std::chrono::time_point<std::chrono::high_resolution_clock> tstart1,
        tstart2,tend1,tend2; // time points

    auto tstart = std::chrono::high_resolution_clock::now(); // start time
        (nano-seconds)

    // Compute seismic impedance
    #pragma omp parallel for
    for (long i=0; i < nlayers; i++)
        imp[i] = density[i] * velocity[i];

    // Reflection coefficients at the base of the layers :
    #pragma omp parallel for
    for (long i=0; i < nlayers-1; i++)
        ref[i] = (imp[i+1] - imp[i])/(imp[i+1] + imp[i]);

    // Spectral window (both low- and high cut)
    #pragma omp parallel for

```

```

for (long i=0; i < lc+1; i++)
half_filter[i]= (sin(M_PI*(2*i-lc)/(2*lc)))/2+0.5;

#pragma omp parallel for
for (long i=0; i < nfreq/2+1; i++)
    filter[i] = half_filter[i];

filter[nfreq/2+1] = 1;

#pragma omp parallel for
for (long i=nfreq/2+2; i < nfreq+1; i++)
    filter[i] = half_filter[nfreq+1-i];

#pragma omp parallel for
for (long i=0; i < n_wave/2; i++)
    half_wave[i] = wave[n_wave/2-1+i];

for (long i=0; i < 2*nfreq; i++) {
    if (i < nfreq) {
        wave_spectral[i] = half_wave[i];
    } else {
        wave_spectral[i] = half_wave[2*nfreq-i];
    }
    mean_wave += std::real(wave_spectral[i]);
}

mean_wave = mean_wave / nsamp;

#pragma omp parallel for
for (long i=0.; i < 2*nfreq; i++)
    wave_spectral[i] -= mean_wave;

// Fourier transform waveform to frequency domain
tstart1 = std::chrono::high_resolution_clock::now(); // start time (
    nano-seconds)
fft(wave_spectral);
tend1 = std::chrono::high_resolution_clock::now(); // end time (nano-
    seconds)

// spectrum U of upgoing waves just below the surface.
// See eq. (43) and (44) in Ganley (1981).

#pragma omp parallel for
for (long i=0; i < nfreq+1; i++) {
    Complex omega{0, 2*M_PI*i*dF};

```

```

    Complex exp_omega = exp( - dT * omega);
    Complex Y = 0;
    for (long n=nlayers-2; n > -1; n--)
        Y = exp_omega * (ref[n] + Y) / (1.0 + ref[n]*Y);
    U[i] = Y;
}

// Compute seismogram
#pragma omp parallel for
for (long i=0; i < nfreq+1; i++) {
    U[i] *= filter[i];
    Upad[i] = U[i];
}

#pragma omp parallel for
for (long i=nfreq+1; i < nsamp; i++)
    Upad[i] = std::conj(Upad[nsamp - i]);

#pragma omp parallel for
for (long i=0; i < nsamp; i++)
    Upad[i] *= wave_spectral[i];

// Fourier transform back again
tstart2 = std::chrono::high_resolution_clock::now(); // start time (
    nano-seconds)
ifft(Upad);
tend2 = std::chrono::high_resolution_clock::now(); // end time (nano-
    seconds)

#pragma omp parallel for
for (long i=0; i < nsamp; i++)
    seismogram[i] = std::real(Upad[i]);

auto tend = std::chrono::high_resolution_clock::now(); // end time (
    nano-seconds)

std::cout << "Wave zero-point          : " << std::setw(9) << std::
    setprecision(5)
    << mean_wave << "\n";
std::cout << "Seismogram first coeff : " << std::setw(9) << std::
    setprecision(5)
    << seismogram[0] << ", " << seismogram[1] << ", " <<
    seismogram[2] << ", " << seismogram[3] << "\n";
std::cout << "Elapsed time for FFTs   : " << std::setw(9) << std::
    setprecision(4)

```

```

        << (tend1 - tstart1 + tend2 - tstart2).count()*1e-9 << "\n";
std::cout << "Elapsed time without FFTs:" << std::setw(9) << std::
    setprecision(4)
        << (tend - tstart - (tend1 - tstart1 + tend2 - tstart2)).
            count()*1e-9 << "\n";
std::cout << "Elapsed time:" << std::setw(9) << std::setprecision(4)
        << (tend - tstart).count()*1e-9 << "\n";

return seismogram;
}

//
=====

//===== Main function
=====
//
=====

int main(int argc, char* argv){
    // Load the wave profile and the density and velocity structure of the
        rock from text files
    DoubleVector wave = read_txt_file("wave_data.txt");           // input
        impulse wave in medium
    DoubleVector density = read_txt_file("density_data.txt");     // density
        as a function of depth
    DoubleVector velocity = read_txt_file("velocity_data.txt");   // seismic
        wave velocity as a function of depth

    // Propagate wave
    DoubleVector seismogram = propagator(wave,density,velocity);

    // write output and make checksum
    double checksum=0;
    std::ofstream file("seismogram.txt"); // open file
    for (long i=0; i < nsamp; i++) {
        file << seismogram[i] << '\n';
        checksum += abs(seismogram[i]);
    }
    std::cout << "Checksum      : " << std::setw(20) << std::setprecision(15)
        << checksum << "\n";
}

```

## 3.2 seismogram\_omp\_fused\_regions.cpp

```

/*
    Assignment: Make an OpenMP parallelised wave propagation
    model for computing the seismic response for a wave
    propagating through a horizontally stratified medium
*/

#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <chrono>
#include <thread>
#include <complex>
#include <cmath>
#include <omp.h>

// shorthand name for complex number type definition below
typedef std::complex<double> Complex;

// =====
// The number of frequencies sets the cost of the problem
#ifndef NFREQ
#define NFREQ (64*1024)
#endif
const long nfreq = NFREQ; // frequencies in spectrum

// =====
// Initialize Basic Constants
const double dT=0.001;      // sampling distance
const long nsamp=2*nfreq;   // samples in seismogram
double dF = 1/(nsamp*dT);   // Frequency resolution (frequency sampling
                             distance)

// =====
// Custom memory allocator for NUMA-aware first touch memory placement
template <class T> class NUMA_Allocator {
public:
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef T& reference;
    typedef const T& const_reference;
    typedef size_t size_type;
    typedef T value_type;

```

```

    NUMA_Allocator() { }
    NUMA_Allocator(const NUMA_Allocator& _r) { }
    ~NUMA_Allocator() { }

    // allocate raw memory including page placement
    pointer allocate(size_t numObjects, const void *localityHint=0) {
        size_t len = numObjects * sizeof(T);
        void *m = std::malloc(len);
        char *p = static_cast<char*>(m);
        if(!omp_in_parallel()) {
#pragma omp parallel for schedule(static)
            for(size_t i=0; i<len; i+=sizeof(T)) {
                for(size_t j=0; j<sizeof(T); ++j) {
                    p[i+j]=0;
                }
            }
        }

        return static_cast<T*>(m);
    }

    // free raw memory
    void deallocate(pointer ptrToMemory, size_t numObjects) {
        std::free(ptrToMemory);
    }

    // construct object at given address
    void construct(pointer p, const value_type& x) {
        new(p) value_type(x);
    }

    // destroy object at given address
    void destroy(pointer p) {
        p->~value_type();
    }

private:
    void operator=(const NUMA_Allocator&) {}
};

// =====
// shorthand names for vector types
#ifdef NUMA_ALLOCATOR
// Use NUMA-aware first touch allocator
typedef std::vector<Complex, NUMA_Allocator<Complex>> ComplexVector;

```

```

typedef std::vector<double, NUMA_Allocator<double>> DoubleVector;
#else
// Use standard allocator
typedef std::vector<Complex> ComplexVector;
typedef std::vector<double> DoubleVector;
#endif

// read data file with one number per line
DoubleVector read_txt_file(std::string fname) {
    DoubleVector data; // vector of data points
    std::string line; // string to read in each line
    std::ifstream file(fname); // open file
    while (std::getline(file, line)) // loop over lines until end of
        file
        data.push_back(std::stod(line)); // convert string to double and
        store in array
    return data;
}

// Cooley Tukey FFT (in-place computation)
template<typename VectorType>
void fft(VectorType& x)
{
    const long N = x.size();
    if (N <= 1) return;

    // divide
    std::vector<Complex> even(N/2), odd(N/2);
    for (long i=0; i<N/2; i++) {
        even[i] = x[2*i];
        odd[i] = x[2*i+1];
    }

    // conquer
    fft(even);
    fft(odd);

    // combine
    for (long k = 0; k < N/2; k++)
    {
        Complex t = std::polar(1.0, -2 * M_PI * k / N) * odd[k];
        x[k] = even[k] + t;
        x[k+N/2] = even[k] - t;
    }
}

```

```

// inverse fft (in-place)
template<typename VectorType>
void ifft(VectorType& x)
{
    double inv_size = 1.0 / x.size();
    for (auto& xx: x) xx = std::conj(xx); // conjugate the input
    fft(x); // forward fft
    for (auto& xx: x)
        xx = std::conj(xx) // conjugate the output
            * inv_size; // scale the numbers
}

// Main routine: propagate wave through layers and compute seismogram
DoubleVector propagator(DoubleVector wave,
                        DoubleVector density,
                        DoubleVector velocity) {
    const long nlayers = density.size();
    DoubleVector imp(nlayers); // impedance
    DoubleVector ref(nlayers-1); // reflection coefficient
    ComplexVector half_filter(nfreq/2+1,1); // half filter
    ComplexVector filter(nfreq+1); // full filter
    DoubleVector half_wave(nfreq+1,0); // half wave
    ComplexVector wave_spectral(nsamp); // FFT(wave)
    ComplexVector U(nfreq+1,0); // Upgoing waves
    ComplexVector Upad(nsamp,0); // FFT(seismogram)
    DoubleVector seismogram(nsamp); // final seismogram
    long n_wave = wave.size(); // size of wave array
    long lc = std::lround(std::floor(nfreq*0.01)); // low-cut indices
    double mean_wave = 0.; // wave zero point
    std::chrono::time_point<std::chrono::high_resolution_clock> tstart1,
        tstart2,tend1,tend2; // time points

    auto tstart = std::chrono::high_resolution_clock::now(); // start time
        (nano-seconds)

    // Compute seismic impedance
    #pragma omp parallel
    {
        #pragma omp for
        for (long i=0; i < nlayers; i++)
            imp[i] = density[i] * velocity[i];

        // Reflection coefficients at the base of the layers :
        #pragma omp for nowait

```

```

for (long i=0; i < nlayers-1; i++)
    ref[i] = (imp[i+1] - imp[i])/(imp[i+1] + imp[i]);

// Spectral window (both low- and high cut)
#pragma omp for
for (long i=0; i < lc+1; i++)
    half_filter[i] = (sin(M_PI*(2*i-lc)/(2*lc)))/2+0.5;

#pragma omp for
for (long i=0; i < nfreq/2+1; i++)
    filter[i] = half_filter[i];

filter[nfreq/2+1] = 1;

#pragma omp for nowait
for (long i=nfreq/2+2; i < nfreq+1; i++)
    filter[i] = half_filter[nfreq+1-i];

#pragma omp for
for (long i=0; i < n_wave/2; i++)
    half_wave[i] = wave[n_wave/2-1+i];

#pragma omp single
for (long i=0; i < 2*nfreq; i++) {
    if (i < nfreq) {
        wave_spectral[i] = half_wave[i];
    } else {
        wave_spectral[i] = half_wave[2*nfreq-i];
    }
    mean_wave += std::real(wave_spectral[i]);
}

#pragma omp single
mean_wave = mean_wave / nsamp;

#pragma omp for
for (long i=0.; i < 2*nfreq; i++)
    wave_spectral[i] -= mean_wave;

#pragma omp single
{
    // Fourier transform waveform to frequency domain
    tstart1 = std::chrono::high_resolution_clock::now(); // start
        time (nano-seconds)
    fft(wave_spectral);

```

```

        tend1 = std::chrono::high_resolution_clock::now(); // end time
                (nano-seconds)
    }

    // spectrum U of upgoing waves just below the surface.
    // See eq. (43) and (44) in Ganley (1981).
    #pragma omp for
    for (long i=0; i < nfreq+1; i++) {
        Complex omega{0, 2*M_PI*i*dF};
        Complex exp_omega = exp( - dT * omega);
        Complex Y = 0;
        for (long n=nlayers-2; n > -1; n--)
            Y = exp_omega * (ref[n] + Y) / (1.0 + ref[n]*Y);
        U[i] = Y;
    }

    // Compute seismogram
    #pragma omp for
    for (long i=0; i < nfreq+1; i++) {
        U[i] *= filter[i];
        Upad[i] = U[i];
    }

    #pragma omp for
    for (long i=nfreq+1; i < nsamp; i++)
        Upad[i] = std::conj(Upad[nsamp - i]);

    #pragma omp for
    for (long i=0; i < nsamp; i++)
        Upad[i] *= wave_spectral[i];

    // Fourier transform back again
    #pragma omp single
    {
        tstart2 = std::chrono::high_resolution_clock::now(); // start time
                (nano-seconds)
        ifft(Upad);
        tend2 = std::chrono::high_resolution_clock::now(); // end time (
                nano-seconds)
    }

    #pragma omp for
    for (long i=0; i < nsamp; i++)
        seismogram[i] = std::real(Upad[i]);
}

```

```

    auto tend = std::chrono::high_resolution_clock::now(); // end time (
        nano-seconds)

    std::cout << "Wave zero-point          : " << std::setw(9) << std::
        setprecision(5)
        << mean_wave << "\n";
    std::cout << "Seismogram first coeff : " << std::setw(9) << std::
        setprecision(5)
        << seismogram[0] << ", " << seismogram[1] << ", " <<
        seismogram[2] << ", " << seismogram[3] << "\n";
    std::cout << "Elapsed time for FFTs   :" << std::setw(9) << std::
        setprecision(4)
        << (tend1 - tstart1 + tend2 - tstart2).count()*1e-9 << "\n";
    std::cout << "Elapsed time without FFTs:" << std::setw(9) << std::
        setprecision(4)
        << (tend - tstart - (tend1 - tstart1 + tend2 - tstart2)).
        count()*1e-9 << "\n";
    std::cout << "Elapsed time:" << std::setw(9) << std::setprecision(4)
        << (tend - tstart).count()*1e-9 << "\n";

    return seismogram;
}

//
=====

//===== Main function
=====
//
=====

int main(int argc, char* argv[]){
    // Load the wave profile and the density and velocity structure of the
        rock from text files
    DoubleVector wave = read_txt_file("wave_data.txt");           // input
        impulse wave in medium
    DoubleVector density = read_txt_file("density_data.txt");     // density
        as a function of depth
    DoubleVector velocity = read_txt_file("velocity_data.txt");   // seismic
        wave velocity as a function of depth

    // Propagate wave
    DoubleVector seismogram = propagator(wave,density,velocity);

    // write output and make checksum

```

```
double checksum=0;
std::ofstream file("seismogram.txt"); // open file
for (long i=0; i < nsamp; i++) {
    file << seismogram[i] << '\n';
    checksum += abs(seismogram[i]);
}
std::cout << "Checksum      : " << std::setw(20) << std::setprecision(15)
          << checksum << "\n";
}
```

### 3.3 seismogram\_omp\_reduction.cpp

```

/*
    Assignment: Make an OpenMP parallelised wave propagation
    model for computing the seismic response for a wave
    propagating through a horizontally stratified medium
*/

#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <chrono>
#include <thread>
#include <complex>
#include <cmath>
#include <omp.h>

// shorthand name for complex number type definition below
typedef std::complex<double> Complex;

// =====
// The number of frequencies sets the cost of the problem
#ifndef NFREQ
#define NFREQ (64*1024)
#endif
const long nfreq = NFREQ; // frequencies in spectrum

// =====
// Initialize Basic Constants
const double dT=0.001;    // sampling distance
const long nsamp=2*nfreq; // samples in seismogram
double dF = 1/(nsamp*dT); // Frequency resolution (frequency sampling
                           distance)

// =====
// Custom memory allocator for NUMA-aware first touch memory placement
template <class T> class NUMA_Allocator {
public:
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef T& reference;
    typedef const T& const_reference;
    typedef size_t size_type;
    typedef T value_type;

```

```

    NUMA_Allocator() { }

    NUMA_Allocator(const NUMA_Allocator& _r) { }

    ~NUMA_Allocator() { }

    // allocate raw memory including page placement
    pointer allocate(size_t numObjects, const void *localityHint=0) {
        size_t len = numObjects * sizeof(T);
        void *m = std::malloc(len);
        char *p = static_cast<char*>(m);
        if(!omp_in_parallel()) {
#pragma omp parallel for schedule(static)
            for(size_t i=0; i<len; i+=sizeof(T)) {
                for(size_t j=0; j<sizeof(T); ++j) {
                    p[i+j]=0;
                }
            }
        }

        return static_cast<T*>(m);
    }

    // free raw memory
    void deallocate(pointer ptrToMemory, size_t numObjects) {
        std::free(ptrToMemory);
    }

    // construct object at given address
    void construct(pointer p, const value_type& x) {
        new(p) value_type(x);
    }

    // destroy object at given address
    void destroy(pointer p) {
        p->~value_type();
    }

private:
    void operator=(const NUMA_Allocator&) {}
};

// =====
// shorthand names for vector types
#ifdef NUMA_ALLOCATOR
// Use NUMA-aware first touch allocator
typedef std::vector<Complex, NUMA_Allocator<Complex>> ComplexVector;

```

```

typedef std::vector<double, NUMA_Allocator<double>> DoubleVector;
#else
// Use standard allocator
typedef std::vector<Complex> ComplexVector;
typedef std::vector<double> DoubleVector;
#endif

// read data file with one number per line
DoubleVector read_txt_file(std::string fname) {
    DoubleVector data; // vector of data points
    std::string line; // string to read in each line
    std::ifstream file(fname); // open file
    while (std::getline(file, line)) // loop over lines until end of
        file
        data.push_back(std::stod(line)); // convert string to double and
        store in array
    return data;
}

// Cooley Tukey FFT (in-place computation)
template<typename VectorType>
void fft(VectorType& x)
{
    const long N = x.size();
    if (N <= 1) return;

    // divide
    std::vector<Complex> even(N/2), odd(N/2);
    for (long i=0; i<N/2; i++) {
        even[i] = x[2*i];
        odd[i] = x[2*i+1];
    }

    // conquer
    fft(even);
    fft(odd);

    // combine
    for (long k = 0; k < N/2; k++)
    {
        Complex t = std::polar(1.0, -2 * M_PI * k / N) * odd[k];
        x[k] = even[k] + t;
        x[k+N/2] = even[k] - t;
    }
}

```

```

// inverse fft (in-place)
template<typename VectorType>
void ifft(VectorType& x)
{
    double inv_size = 1.0 / x.size();
    for (auto& xx: x) xx = std::conj(xx); // conjugate the input
    fft(x); // forward fft
    for (auto& xx: x)
        xx = std::conj(xx) // conjugate the output
            * inv_size; // scale the numbers
}

// Main routine: propagate wave through layers and compute seismogram
DoubleVector propagator(DoubleVector wave,
                        DoubleVector density,
                        DoubleVector velocity) {
    const long nlayers = density.size();
    DoubleVector imp(nlayers); // impedance
    DoubleVector ref(nlayers-1); // reflection coefficient
    ComplexVector half_filter(nfreq/2+1,1); // half filter
    ComplexVector filter(nfreq+1); // full filter
    DoubleVector half_wave(nfreq+1,0); // half wave
    ComplexVector wave_spectral(nsamp); // FFT(wave)
    ComplexVector U(nfreq+1,0); // Upgoing waves
    ComplexVector Upad(nsamp,0); // FFT(seismogram)
    DoubleVector seismogram(nsamp); // final seismogram
    long n_wave = wave.size(); // size of wave array
    long lc = std::lround(std::floor(nfreq*0.01)); // low-cut indices
    double mean_wave = 0.; // wave zero point
    std::chrono::time_point<std::chrono::high_resolution_clock> tstart1,
        tstart2,tend1,tend2; // time points

    auto tstart = std::chrono::high_resolution_clock::now(); // start time
        (nano-seconds)

    // Compute seismic impedance
    #pragma omp parallel
    {
        #pragma omp for
        for (long i=0; i < nlayers; i++)
            imp[i] = density[i] * velocity[i];

        // Reflection coefficients at the base of the layers :
        #pragma omp for nowait

```

```

for (long i=0; i < nlayers-1; i++)
    ref[i] = (imp[i+1] - imp[i])/(imp[i+1] + imp[i]);

// Spectral window (both low- and high cut)
#pragma omp for
for (long i=0; i < lc+1; i++)
    half_filter[i] = (sin(M_PI*(2*i-lc)/(2*lc)))/2+0.5;

#pragma omp for
for (long i=0; i < nfreq/2+1; i++)
    filter[i] = half_filter[i];

filter[nfreq/2+1] = 1;

#pragma omp for nowait
for (long i=nfreq/2+2; i < nfreq+1; i++)
    filter[i] = half_filter[nfreq+1-i];

#pragma omp for
for (long i=0; i < n_wave/2; i++)
    half_wave[i] = wave[n_wave/2-1+i];

#pragma omp for reduction(+:mean_wave)
for (long i=0; i < 2*nfreq; i++) {
    if (i < nfreq) {
        wave_spectral[i] = half_wave[i];
    } else {
        wave_spectral[i] = half_wave[2*nfreq-i];
    }
    mean_wave += std::real(wave_spectral[i]);
}

#pragma omp single
mean_wave = mean_wave / nsamp;

#pragma omp for
for (long i=0.; i < 2*nfreq; i++)
    wave_spectral[i] -= mean_wave;

#pragma omp single
{
    // Fourier transform waveform to frequency domain
    tstart1 = std::chrono::high_resolution_clock::now(); // start
    time (nano-seconds)
    fft(wave_spectral);

```

```

        tend1 = std::chrono::high_resolution_clock::now(); // end time
                (nano-seconds)
    }

    // spectrum U of upgoing waves just below the surface.
    // See eq. (43) and (44) in Ganley (1981).
    #pragma omp for
    for (long i=0; i < nfreq+1; i++) {
        Complex omega{0, 2*M_PI*i*dF};
        Complex exp_omega = exp( - dT * omega);
        Complex Y = 0;
        for (long n=nlayers-2; n > -1; n--)
            Y = exp_omega * (ref[n] + Y) / (1.0 + ref[n]*Y);
        U[i] = Y;
    }

    // Compute seismogram
    #pragma omp for
    for (long i=0; i < nfreq+1; i++) {
        U[i] *= filter[i];
        Upad[i] = U[i];
    }

    #pragma omp for
    for (long i=nfreq+1; i < nsamp; i++)
        Upad[i] = std::conj(Upad[nsamp - i]);

    #pragma omp for
    for (long i=0; i < nsamp; i++)
        Upad[i] *= wave_spectral[i];

    // Fourier transform back again
    #pragma omp single
    {
        tstart2 = std::chrono::high_resolution_clock::now(); // start time
                (nano-seconds)
        ifft(Upad);
        tend2 = std::chrono::high_resolution_clock::now(); // end time (
                nano-seconds)
    }

    #pragma omp for
    for (long i=0; i < nsamp; i++)
        seismogram[i] = std::real(Upad[i]);
}

```

```

    auto tend = std::chrono::high_resolution_clock::now(); // end time (
        nano-seconds)

    std::cout << "Wave zero-point          : " << std::setw(9) << std:::
        setprecision(5)
        << mean_wave << "\n";
    std::cout << "Seismogram first coeff : " << std::setw(9) << std:::
        setprecision(5)
        << seismogram[0] << ", " << seismogram[1] << ", " <<
        seismogram[2] << ", " << seismogram[3] << "\n";
    std::cout << "Elapsed time for FFTs   :" << std::setw(9) << std:::
        setprecision(4)
        << (tend1 - tstart1 + tend2 - tstart2).count()*1e-9 << "\n";
    std::cout << "Elapsed time without FFTs:" << std::setw(9) << std:::
        setprecision(4)
        << (tend - tstart - (tend1 - tstart1 + tend2 - tstart2)).
        count()*1e-9 << "\n";
    std::cout << "Elapsed time:" << std::setw(9) << std::setprecision(4)
        << (tend - tstart).count()*1e-9 << "\n";

    return seismogram;
}

//
=====

//===== Main function
=====
//
=====

int main(int argc, char* argv[]){
    // Load the wave profile and the density and velocity structure of the
        rock from text files
    DoubleVector wave = read_txt_file("wave_data.txt");           // input
        impulse wave in medium
    DoubleVector density = read_txt_file("density_data.txt");     // density
        as a function of depth
    DoubleVector velocity = read_txt_file("velocity_data.txt");   // seismic
        wave velocity as a function of depth

    // Propagate wave
    DoubleVector seismogram = propagator(wave,density,velocity);

    // write output and make checksum

```

```
double checksum=0;
std::ofstream file("seismogram.txt"); // open file
for (long i=0; i < nsamp; i++) {
    file << seismogram[i] << '\n';
    checksum += abs(seismogram[i]);
}
std::cout << "Checksum      : " << std::setw(20) << std::setprecision(15)
           << checksum << "\n";
}
```

## 3.4 seismogram\_omp\_SIMD.cpp

```

/*
    Assignment: Make an OpenMP parallelised wave propagation
    model for computing the seismic response for a wave
    propagating through a horizontally stratified medium
*/

#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <chrono>
#include <thread>
#include <complex>
#include <cmath>
#include <omp.h>

// shorthand name for complex number type definition below
typedef std::complex<double> Complex;

// =====
// The number of frequencies sets the cost of the problem
#ifndef NFREQ
#define NFREQ (64*1024)
#endif
const long nfreq = NFREQ; // frequencies in spectrum

// =====
// Initialize Basic Constants
const double dT=0.001;      // sampling distance
const long nsamp=2*nfreq;   // samples in seismogram
double dF = 1/(nsamp*dT);   // Frequency resolution (frequency sampling
                             distance)

// =====
// Custom memory allocator for NUMA-aware first touch memory placement
template <class T> class NUMA_Allocator {
public:
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef T& reference;
    typedef const T& const_reference;
    typedef size_t size_type;
    typedef T value_type;

```

```

    NUMA_Allocator() { }

    NUMA_Allocator(const NUMA_Allocator& _r) { }

    ~NUMA_Allocator() { }

    // allocate raw memory including page placement
    pointer allocate(size_t numObjects, const void *localityHint=0) {
        size_t len = numObjects * sizeof(T);
        void *m = std::malloc(len);
        char *p = static_cast<char*>(m);
        if(!omp_in_parallel()) {
#pragma omp parallel for schedule(static)
            for(size_t i=0; i<len; i+=sizeof(T)) {
                for(size_t j=0; j<sizeof(T); ++j) {
                    p[i+j]=0;
                }
            }
        }

        return static_cast<T*>(m);
    }

    // free raw memory
    void deallocate(pointer ptrToMemory, size_t numObjects) {
        std::free(ptrToMemory);
    }

    // construct object at given address
    void construct(pointer p, const value_type& x) {
        new(p) value_type(x);
    }

    // destroy object at given address
    void destroy(pointer p) {
        p->~value_type();
    }

private:
    void operator=(const NUMA_Allocator&) {}
};

// =====
// shorthand names for vector types
#ifdef NUMA_ALLOCATOR
// Use NUMA-aware first touch allocator
typedef std::vector<Complex, NUMA_Allocator<Complex>> ComplexVector;

```

```

typedef std::vector<double, NUMA_Allocator<double>> DoubleVector;
#else
// Use standard allocator
typedef std::vector<Complex> ComplexVector;
typedef std::vector<double> DoubleVector;
#endif

// read data file with one number per line
DoubleVector read_txt_file(std::string fname) {
    DoubleVector data; // vector of data points
    std::string line; // string to read in each line
    std::ifstream file(fname); // open file
    while (std::getline(file, line)) // loop over lines until end of
        file
        data.push_back(std::stod(line)); // convert string to double and
        store in array
    return data;
}

// Cooley Tukey FFT (in-place computation)
template<typename VectorType>
void fft(VectorType& x)
{
    const long N = x.size();
    if (N <= 1) return;

    // divide
    std::vector<Complex> even(N/2), odd(N/2);
    for (long i=0; i<N/2; i++) {
        even[i] = x[2*i];
        odd[i] = x[2*i+1];
    }

    // conquer
    fft(even);
    fft(odd);

    // combine
    for (long k = 0; k < N/2; k++)
    {
        Complex t = std::polar(1.0, -2 * M_PI * k / N) * odd[k];
        x[k] = even[k] + t;
        x[k+N/2] = even[k] - t;
    }
}

```

```

}

// inverse fft (in-place)
template<typename VectorType>
void ifft(VectorType& x)
{
    double inv_size = 1.0 / x.size();
    for (auto& xx: x) xx = std::conj(xx); // conjugate the input

    fft(x); // forward fft

    for (auto& xx: x)
        xx = std::conj(xx) // conjugate the output
            * inv_size; // scale the numbers
}

// Main routine: propagate wave through layers and compute seismogram
DoubleVector propagator(DoubleVector wave,
                        DoubleVector density,
                        DoubleVector velocity) {
    const long nlayers = density.size();
    DoubleVector imp(nlayers); // impedance
    DoubleVector ref(nlayers-1); // reflection coefficient
    ComplexVector half_filter(nfreq/2+1,1); // half filter
    ComplexVector filter(nfreq+1); // full filter
    DoubleVector half_wave(nfreq+1,0); // half wave
    ComplexVector wave_spectral(nsamp); // FFT(wave)
    ComplexVector U(nfreq+1,0); // Upgoing waves
    ComplexVector Upad(nsamp,0); // FFT(seismogram)
    DoubleVector seismogram(nsamp); // final seismogram
    long n_wave = wave.size(); // size of wave array
    long lc = std::lround(std::floor(nfreq*0.01)); // low-cut indices
    double mean_wave = 0.; // wave zero point
    std::chrono::time_point<std::chrono::high_resolution_clock> tstart1,
        tstart2,tend1,tend2; // time points

    auto tstart = std::chrono::high_resolution_clock::now(); // start time
        (nano-seconds)

    // Compute seismic impedance
    #pragma omp parallel
    {
        #pragma omp for simd
        for (long i=0; i < nlayers; i++)
            imp[i] = density[i] * velocity[i];
    }
}

```

```

// Reflection coefficients at the base of the layers :
#pragma omp for simd nowait
for (long i=0; i < nlayers-1; i++)
    ref[i] = (imp[i+1] - imp[i])/(imp[i+1] + imp[i]);

// Spectral window (both low- and high cut)
#pragma omp for simd
for (long i=0; i < lc+1; i++)
    half_filter[i] = (sin(M_PI*(2*i-lc)/(2*lc)))/2+0.5;

#pragma omp for simd
for (long i=0; i < nfreq/2+1; i++)
    filter[i] = half_filter[i];

filter[nfreq/2+1] = 1;

#pragma omp for simd nowait
for (long i=nfreq/2+2; i < nfreq+1; i++)
    filter[i] = half_filter[nfreq+1-i];

#pragma omp for simd
for (long i=0; i < n_wave/2; i++)
    half_wave[i] = wave[n_wave/2-1+i];

#pragma omp for reduction(+:mean_wave)
for (long i=0; i < 2*nfreq; i++) {
    if (i < nfreq) {
        wave_spectral[i] = half_wave[i];
    } else {
        wave_spectral[i] = half_wave[2*nfreq-i];
    }
    mean_wave += std::real(wave_spectral[i]);
}

#pragma omp single
mean_wave = mean_wave / nsamp;

#pragma omp for simd
for (long i=0.; i < 2*nfreq; i++)
    wave_spectral[i] -= mean_wave;

#pragma omp single
{
    // Fourier transform waveform to frequency domain

```

```

        tstart1 = std::chrono::high_resolution_clock::now(); // start
            time (nano-seconds)
        fft(wave_spectral);
        tend1 = std::chrono::high_resolution_clock::now(); // end time
            (nano-seconds)
    }

// spectrum U of upgoing waves just below the surface.
// See eq. (43) and (44) in Ganley (1981).
#pragma omp for simd
for (long i=0; i < nfreq+1; i++) {
    Complex omega{0, 2*M_PI*i*dF};
    Complex exp_omega = exp( - dT * omega);
    Complex Y = 0;
    for (long n=nlayers-2; n > -1; n--)
        Y = exp_omega * (ref[n] + Y) / (1.0 + ref[n]*Y);
    U[i] = Y;
}

// Compute seismogram
#pragma omp for simd
for (long i=0; i < nfreq+1; i++) {
    U[i] *= filter[i];
    Upad[i] = U[i];
}

#pragma omp for simd
for (long i=nfreq+1; i < nsamp; i++)
    Upad[i] = std::conj(Upad[nsamp - i]);

#pragma omp for simd
for (long i=0; i < nsamp; i++)
    Upad[i] *= wave_spectral[i];

// Fourier transform back again
#pragma omp single
{
    tstart2 = std::chrono::high_resolution_clock::now(); // start time
        (nano-seconds)
    ifft(Upad);
    tend2 = std::chrono::high_resolution_clock::now(); // end time (
        nano-seconds)
}

#pragma omp for simd

```

```

        for (long i=0; i < nsamp; i++)
            seismogram[i] = std::real(Upad[i]);
    }

    auto tend = std::chrono::high_resolution_clock::now(); // end time (
        nano-seconds)

    std::cout << "Wave zero-point          : " << std::setw(9) << std::
        setprecision(5)
        << mean_wave << "\n";
    std::cout << "Seismogram first coeff : " << std::setw(9) << std::
        setprecision(5)
        << seismogram[0] << ", " << seismogram[1] << ", " <<
        seismogram[2] << ", " << seismogram[3] << "\n";
    std::cout << "Elapsed time for FFTs   :" << std::setw(9) << std::
        setprecision(4)
        << (tend1 - tstart1 + tend2 - tstart2).count()*1e-9 << "\n";
    std::cout << "Elapsed time without FFTs:" << std::setw(9) << std::
        setprecision(4)
        << (tend - tstart - (tend1 - tstart1 + tend2 - tstart2)).
        count()*1e-9 << "\n";
    std::cout << "Elapsed time:" << std::setw(9) << std::setprecision(4)
        << (tend - tstart).count()*1e-9 << "\n";

    return seismogram;
}

//
=====

//===== Main function
=====
//
=====

int main(int argc, char* argv[]){
    // Load the wave profile and the density and velocity structure of the
        rock from text files
    DoubleVector wave = read_txt_file("wave_data.txt");           // input
        impulse wave in medium
    DoubleVector density = read_txt_file("density_data.txt");     // density
        as a function of depth
    DoubleVector velocity = read_txt_file("velocity_data.txt");   // seismic
        wave velocity as a function of depth

    // Propagate wave

```

```
DoubleVector seismogram = propagator(wave,density,velocity);

// write output and make checksum
double checksum=0;
std::ofstream file("seismogram.txt"); // open file
for (long i=0; i < nsamp; i++) {
    file << seismogram[i] << '\n';
    checksum += abs(seismogram[i]);
}
std::cout << "Checksum      : " << std::setw(20) << std::setprecision(15)
    << checksum << "\n";
}
```

### 3.5 seismogram\_omp\_FFT.cpp

```

/*
    Assignment: Make an OpenMP parallelised wave propagation
    model for computing the seismic response for a wave
    propagating through a horizontally stratified medium
*/

#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <chrono>
#include <thread>
#include <complex>
#include <cmath>
#include <omp.h>

// shorthand name for complex number type definition below
typedef std::complex<double> Complex;

// =====
// The number of frequencies sets the cost of the problem
#ifndef NFREQ
#define NFREQ (64*1024)
#endif
const long nfreq = NFREQ; // frequencies in spectrum

// =====
// Initialize Basic Constants
const double dT=0.001;    // sampling distance
const long nsamp=2*nfreq; // samples in seismogram
double dF = 1/(nsamp*dT); // Frequency resolution (frequency sampling
                           distance)

// =====
// Custom memory allocator for NUMA-aware first touch memory placement
template <class T> class NUMA_Allocator {
public:
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef T& reference;
    typedef const T& const_reference;
    typedef size_t size_type;
    typedef T value_type;

```

```

    NUMA_Allocator() { }

    NUMA_Allocator(const NUMA_Allocator& _r) { }

    ~NUMA_Allocator() { }

    // allocate raw memory including page placement
    pointer allocate(size_t numObjects, const void *localityHint=0) {
        size_t len = numObjects * sizeof(T);
        void *m = std::malloc(len);
        char *p = static_cast<char*>(m);
        if(!omp_in_parallel()) {
#pragma omp parallel for schedule(static)
            for(size_t i=0; i<len; i+=sizeof(T)) {
                for(size_t j=0; j<sizeof(T); ++j) {
                    p[i+j]=0;
                }
            }
        }

        return static_cast<T*>(m);
    }

    // free raw memory
    void deallocate(pointer ptrToMemory, size_t numObjects) {
        std::free(ptrToMemory);
    }

    // construct object at given address
    void construct(pointer p, const value_type& x) {
        new(p) value_type(x);
    }

    // destroy object at given address
    void destroy(pointer p) {
        p->~value_type();
    }

private:
    void operator=(const NUMA_Allocator&) {}
};

// =====
// shorthand names for vector types
#ifdef NUMA_ALLOCATOR
// Use NUMA-aware first touch allocator
typedef std::vector<Complex, NUMA_Allocator<Complex>> ComplexVector;

```

```

typedef std::vector<double, NUMA_Allocator<double>> DoubleVector;
#else
// Use standard allocator
typedef std::vector<Complex> ComplexVector;
typedef std::vector<double> DoubleVector;
#endif

// read data file with one number per line
DoubleVector read_txt_file(std::string fname) {
    DoubleVector data; // vector of data points
    std::string line; // string to read in each line
    std::ifstream file(fname); // open file
    while (std::getline(file, line)) // loop over lines until end of
        file
        data.push_back(std::stod(line)); // convert string to double and
        store in array
    return data;
}

// Cooley Tukey FFT (in-place computation)
template<typename VectorType>
void fft(VectorType& x)
{
    const long N = x.size();
    if (N <= 1) return;

    // divide
    std::vector<Complex> even(N/2), odd(N/2);
    for (long i=0; i<N/2; i++) {
        even[i] = x[2*i];
        odd[i] = x[2*i+1];
    }

    // conquer
    if (N > 1024) { // threshold
        #pragma omp task shared(even)
        fft(even);

        #pragma omp task shared(odd)
        fft(odd);

        #pragma omp taskwait
    } else {
        fft(even);
        fft(odd);
    }
}

```

```

    }

    // combine
    for (long k = 0; k < N/2; k++)
    {
        Complex t = std::polar(1.0, -2 * M_PI * k / N) * odd[k];
        x[k] = even[k] + t;
        x[k+N/2] = even[k] - t;
    }
}

// inverse fft (in-place)
template<typename VectorType>
void ifft(VectorType& x)
{
    double inv_size = 1.0 / x.size();
    for (auto& xx: x) xx = std::conj(xx); // conjugate the input

    fft(x); // forward fft

    for (auto& xx: x)
        xx = std::conj(xx) // conjugate the output
            * inv_size; // scale the numbers
}

// Main routine: propagate wave through layers and compute seismogram
DoubleVector propagator(DoubleVector wave,
                        DoubleVector density,
                        DoubleVector velocity) {
    const long nlayers = density.size();
    DoubleVector imp(nlayers); // impedance
    DoubleVector ref(nlayers-1); // reflection coefficient
    ComplexVector half_filter(nfreq/2+1,1); // half filter
    ComplexVector filter(nfreq+1); // full filter
    DoubleVector half_wave(nfreq+1,0); // half wave
    ComplexVector wave_spectral(nsamp); // FFT(wave)
    ComplexVector U(nfreq+1,0); // Upgoing waves
    ComplexVector Upad(nsamp,0); // FFT(seismogram)
    DoubleVector seismogram(nsamp); // final seismogram
    long n_wave = wave.size(); // size of wave array
    long lc = std::lround(std::floor(nfreq*0.01)); // low-cut indices
    double mean_wave = 0.; // wave zero point
    std::chrono::time_point<std::chrono::high_resolution_clock> tstart1,
        tstart2,tend1,tend2; // time points

```

```

auto tstart = std::chrono::high_resolution_clock::now(); // start time
(nano-seconds)

// Compute seismic impedance
#pragma omp parallel
{
    #pragma omp for simd
    for (long i=0; i < nlayers; i++)
        imp[i] = density[i] * velocity[i];

    // Reflection coefficients at the base of the layers :
    #pragma omp for simd nowait
    for (long i=0; i < nlayers-1; i++)
        ref[i] = (imp[i+1] - imp[i])/(imp[i+1] + imp[i]);

    // Spectral window (both low- and high cut)
    #pragma omp for simd
    for (long i=0; i < lc+1; i++)
        half_filter[i] = (sin(M_PI*(2*i-lc)/(2*lc)))/2+0.5;

    #pragma omp for simd
    for (long i=0; i < nfreq/2+1; i++)
        filter[i] = half_filter[i];

    filter[nfreq/2+1] = 1;

    #pragma omp for simd nowait
    for (long i=nfreq/2+2; i < nfreq+1; i++)
        filter[i] = half_filter[nfreq+1-i];

    #pragma omp for simd
    for (long i=0; i < n_wave/2; i++)
        half_wave[i] = wave[n_wave/2-1+i];

    #pragma omp for reduction(+:mean_wave)
    for (long i=0; i < 2*nfreq; i++) {
        if (i < nfreq) {
            wave_spectral[i] = half_wave[i];
        } else {
            wave_spectral[i] = half_wave[2*nfreq-i];
        }
        mean_wave += std::real(wave_spectral[i]);
    }
}

```

```

#pragma omp single
mean_wave = mean_wave / nsamp;

#pragma omp for simd
for (long i=0.; i < 2*nfreq; i++)
    wave_spectral[i] -= mean_wave;

#pragma omp single
{
    // Fourier transform waveform to frequency domain
    tstart1 = std::chrono::high_resolution_clock::now(); // start
        time (nano-seconds)
    fft(wave_spectral);
    tend1 = std::chrono::high_resolution_clock::now(); // end time
        (nano-seconds)
}

// spectrum U of upgoing waves just below the surface.
// See eq. (43) and (44) in Ganley (1981).
#pragma omp for simd
for (long i=0; i < nfreq+1; i++) {
    Complex omega{0, 2*M_PI*i*dF};
    Complex exp_omega = exp( - dT * omega);
    Complex Y = 0;
    for (long n=nlayers-2; n > -1; n--)
        Y = exp_omega * (ref[n] + Y) / (1.0 + ref[n]*Y);
    U[i] = Y;
}

// Compute seismogram
#pragma omp for simd
for (long i=0; i < nfreq+1; i++) {
    U[i] *= filter[i];
    Upad[i] = U[i];
}

#pragma omp for simd
for (long i=nfreq+1; i < nsamp; i++)
    Upad[i] = std::conj(Upad[nsamp - i]);

#pragma omp for simd
for (long i=0; i < nsamp; i++)
    Upad[i] *= wave_spectral[i];

// Fourier transform back again

```

```

#pragma omp single
{
    tstart2 = std::chrono::high_resolution_clock::now(); // start time
        (nano-seconds)
    ifft(Upad);
    tend2 = std::chrono::high_resolution_clock::now(); // end time (
        nano-seconds)
}

#pragma omp for simd
for (long i=0; i < nsamp; i++)
    seismogram[i] = std::real(Upad[i]);
}

auto tend = std::chrono::high_resolution_clock::now(); // end time (
    nano-seconds)

std::cout << "Wave zero-point      : " << std::setw(9) << std::
    setprecision(5)
        << mean_wave << "\n";
std::cout << "Seismogram first coeff : " << std::setw(9) << std::
    setprecision(5)
        << seismogram[0] << ", " << seismogram[1] << ", " <<
            seismogram[2] << ", " << seismogram[3] << "\n";
std::cout << "Elapsed time for FFTs  :" << std::setw(9) << std::
    setprecision(4)
        << (tend1 - tstart1 + tend2 - tstart2).count()*1e-9 << "\n";
std::cout << "Elapsed time without FFTs:" << std::setw(9) << std::
    setprecision(4)
        << (tend - tstart - (tend1 - tstart1 + tend2 - tstart2)).
            count()*1e-9 << "\n";
std::cout << "Elapsed time:" << std::setw(9) << std::setprecision(4)
        << (tend - tstart).count()*1e-9 << "\n";

return seismogram;
}

//
=====

//===== Main function
=====
//
=====

int main(int argc, char* argv[]){

```

```

// Load the wave profile and the density and velocity structure of the
    rock from text files
DoubleVector wave = read_txt_file("wave_data.txt");           // input
    impulse wave in medium
DoubleVector density = read_txt_file("density_data.txt");     // density
    as a function of depth
DoubleVector velocity = read_txt_file("velocity_data.txt"); // seismic
    wave velocity as a function of depth

// Propagate wave
DoubleVector seismogram = propagator(wave,density,velocity);

// write output and make checksum
double checksum=0;
std::ofstream file("seismogram.txt"); // open file
for (long i=0; i < nsamp; i++) {
    file << seismogram[i] << '\n';
    checksum += abs(seismogram[i]);
}
std::cout << "Checksum      : " << std::setw(20) << std::setprecision(15)
    << checksum << "\n";
}

```